MASTER
GIT & GITHUB

Command Reference: every command from the masterclass, explained
From basics to advanced workflows

How to read this guide

Commands are in `red monospace`. Anything in `<angle brackets>` is a placeholder you replace with your own value (for example `<branch>` becomes `feature/login`). Sections run in the same order as the class: basics first, advanced tools last.

The mental model in 30 seconds

*Git moves your work through **four places**. Almost every command is about moving changes from one place to the next, or back.*

1. Working directory	2. Staging area	3. Local repository	4. Remote (GitHub)
The files you edit on your computer.	A holding area for what goes into the next commit.	Your full history of commits, stored in the hidden .git folder.	The shared copy on GitHub that your team uses.
`git add` moves changes right to staging	`git commit` saves staging into history	`git push` uploads history to GitHub	`git pull` / `fetch` bring changes back

Contents

1. Setup and configuration
2. Creating and cloning repositories
3. Everyday snapshotting (add, commit)
4. Inspecting history and changes
5. Undoing mistakes
6. Branching
7. Merging and resolving conflicts
8. Rebasing
9. Remotes: fetch, pull, push
10. Stash
11. Cherry-pick
12. Reflog and recovery
13. Tags
14. More power tools (bisect, blame, worktree...)
15. GitHub collaboration and the gh CLI
16. GitHub Actions starter
17. Team workflows: Git Flow vs trunk-based
18. Good commit messages
19. Panic button: common errors and fixes
20. Golden rules

1. Setup and configuration

Do this once per computer so every commit carries your name and email.

Command	What it does
`git --version`	Shows the installed Git version. Use it to confirm Git is installed.
`git config --global user.name "Your Name"`	Sets the name attached to every commit you make.
`git config --global user.email "you@example.com"`	Sets the email attached to your commits. Use the same email as your GitHub account so commits link to you.
`git config --global init.defaultBranch main`	Makes new repositories start with a branch called **main** instead of master.
`git config --global core.editor "code --wait"`	Uses VS Code as the editor when Git needs you to type a message.
`git config --list`	Prints every Git setting currently in effect.
`git config --global alias.st status`	Creates a shortcut: after this, **git st** runs **git status**.
`git help <command>`	Opens the manual for a command. **git <command> -h** gives a short summary.

2. Creating and cloning repositories

Start a new project, or download an existing one.

Command	What it does
`git init`	Turns the current folder into a Git repository by creating a hidden **.git** folder.
`git init <folder>`	Creates a new folder and initialises a repository inside it.
`git clone <url>`	Downloads a full copy of a remote repository, including all history, and sets it up as **origin**.
`git clone <url> <folder>`	Same as above, but into a folder name you choose.
`git clone --depth 1 <url>`	Shallow clone: downloads only the latest snapshot. Much faster for large repos.
`git clone -b <branch> <url>`	Clones and immediately checks out a specific branch.

3. Everyday snapshotting

The loop you will repeat hundreds of times: change, stage, commit.

Command	What it does
`git status`	Shows what changed, what is staged, and which files are untracked. Run it constantly.
`git status -s`	Short, compact version of status.
`git add <file>`	Stages one file so it goes into the next commit.
`git add .`	Stages all changes in the current folder and below.
`git add -A`	Stages every change in the whole repository, including deletions.
`git add -p`	Interactive mode: review each chunk of change and choose to stage it or not. Great for clean, focused commits.
`git commit -m "message"`	Saves everything staged as a new commit with your message.

Command	What it does
`git commit -am "message"`	Stages all modified *tracked* files and commits in one step. It does **not** include brand-new files.
`git commit`	Opens your editor so you can write a longer, multi-line commit message.
`git commit --amend`	Rewrites the **last** commit: change its message or add forgotten staged files. Only do this before pushing.
`git commit --amend --no-edit`	Adds staged changes to the last commit and keeps its message unchanged.
`git rm <file>`	Deletes a file and stages the deletion.
`git rm --cached <file>`	Stops tracking a file but keeps it on disk. Use it for files that should have been in .gitignore.
`git mv <old> <new>`	Renames or moves a file and stages the change.
`.gitignore (a file)`	A text file listing patterns Git should never track, such as **node_modules/**, **.env**, ***.log**.

4. Inspecting history and changes

Look around before you act. These commands never change anything.

Command	What it does
`git log`	Shows the commit history, newest first.
`git log --oneline`	One line per commit: short hash plus message.
`git log --oneline --graph --all`	Draws the branch and merge structure as an ASCII graph. The best way to *see* branches.
`git log -p`	Shows each commit together with the exact lines changed.
`git log --stat`	Shows which files each commit touched and how many lines changed.
`git log --author="name"`	Filters history to one author.
`git log --since="2 weeks ago"`	Shows only commits from a time range.
`git log -- <file>`	Shows the history of a single file.
`git show <commit>`	Shows one commit in detail: message, author, and the changes.
`git diff`	Shows changes in your working directory that are **not yet staged**.
`git diff --staged`	Shows what is staged and will go into the next commit.
`git diff <branchA> <branchB>`	Compares two branches or two commits.
`git blame <file>`	Shows who last changed each line of a file, and in which commit.
`git shortlog -sn`	Counts commits per author. A quick contributor leaderboard.

5. Undoing mistakes

Git is forgiving. Pick the right undo for the situation. Items marked DANGER discard work.

Command	What it does
`git restore <file>`	Throws away your uncommitted edits to a file and returns it to the last commit. **DANGER:** edits are lost.
`git restore --staged <file>`	Unstages a file but keeps your edits. The reverse of **git add**.
`git restore --source=<commit> <file>`	Brings back a file exactly as it was in an older commit.
`git reset --soft HEAD~1`	Undoes the last commit but keeps all its changes **staged**.
`git reset HEAD~1`	Undoes the last commit and keeps its changes in your files, **unstaged** (this is the default, --mixed).
`git reset --hard HEAD~1`	Undoes the last commit and **deletes** its changes. **DANGER** (recoverable only through reflog).
`git revert <commit>`	Creates a *new* commit that reverses an earlier one. Safe for shared history because nothing is rewritten.
`git clean -n`	Dry run: lists untracked files that **would** be deleted.
`git clean -fd`	Deletes untracked files and folders. **DANGER:** always run **-n** first.

6. Branching

A branch is just a movable pointer to a commit. Branches are cheap, so use them for every task.

Command	What it does
`git branch`	Lists local branches. The current one is marked with a star.
`git branch -a`	Lists local and remote branches.
`git branch -vv`	Lists branches with their last commit and which remote branch they track.
`git branch <name>`	Creates a branch but does not switch to it.
`git switch <name>`	Moves you onto an existing branch.
`git switch -c <name>`	Creates a new branch and switches to it in one step.
`git checkout <name>`	The older command that does the same as switch. You will see it in many tutorials.
`git checkout -b <name>`	Older equivalent of **git switch -c**.
`git branch -m <new-name>`	Renames the current branch.
`git branch -d <name>`	Deletes a branch that is already merged. Git refuses if it is not.
`git branch -D <name>`	Force-deletes a branch even if unmerged. **DANGER.**
`git push origin --delete <name>`	Deletes the branch on the remote (GitHub).

7. Merging and resolving conflicts

Bring the work of one branch into another.

Command	What it does
`git merge <branch>`	Merges **<branch>** into the branch you are on. Git does a fast-forward if possible, otherwise a merge commit.
`git merge --no-ff <branch>`	Always creates a merge commit, which keeps the record that a feature branch existed.
`git merge --squash <branch>`	Combines all of a branch's commits into one set of staged changes. You then commit it yourself.
`git merge --abort`	Cancels a merge that hit conflicts and returns to how things were before.
`git status`	During a conflict, lists the files marked **both modified** that need your attention.
`git diff --name-only --diff-filter=U`	Prints only the files that are still unresolved.
`git mergetool`	Opens your configured visual merge tool to resolve conflicts.
`git add <file> then git commit`	After editing a conflicted file and removing the markers, stage it and commit to finish the merge.

How to read conflict markers

```
<<<<<<< HEAD
  your current branch's version of the lines
=======
  the incoming branch's version of the lines
>>>>>>> feature/login

# Edit the file to keep what you want, delete the three marker lines,
# then:  git add <file>   and   git commit
```

8. Rebasing

Rebase replays your commits on top of another branch to give a clean, straight history.

Command	What it does
`git rebase main`	Moves your current branch's commits so they sit on top of the latest **main**. Rewrites those commits.
`git rebase -i HEAD~3`	Interactive rebase of the last 3 commits: reword, squash, reorder, edit, or drop them.
`git rebase --continue`	After fixing a conflict mid-rebase, stage the fix then run this to carry on.
`git rebase --skip`	Skips the commit that is causing trouble.
`git rebase --abort`	Cancels the rebase and returns to the state before it started.
`git pull --rebase`	Fetches, then replays your local commits on top of the remote ones instead of creating a merge commit.
`git commit --fixup <commit>`	Marks a commit as a fix for an earlier one. Pair with **git rebase -i --autosquash** to fold it in automatically.

Golden rule of rebasing: never rebase commits that other people already have (public or shared branches). Rebase your own local work, merge shared work.

9. Remotes: fetch, pull, push

Connect your local repository to GitHub and keep both in sync.

Command	What it does
`git remote -v`	Lists your remotes and their URLs.
`git remote add origin <url>`	Connects your local repo to a remote and names it **origin**.
`git remote set-url origin <url>`	Changes the URL of an existing remote.
`git remote remove <name>`	Removes a remote connection.
`git fetch`	Downloads new commits from the remote but does **not** change your files. Safe to run any time.
`git fetch --all --prune`	Fetches from all remotes and cleans up references to branches deleted on the remote.
`git pull`	Fetch plus merge: downloads and immediately integrates remote changes into your current branch.
`git push`	Uploads your commits to the tracked remote branch.
`git push -u origin <branch>`	Pushes a new branch and links it to the remote one, so later you can type just **git push** or **git pull**.
`git push --force-with-lease`	A safer force push: it only overwrites the remote if nobody else has pushed since you last fetched. Use after rebasing your own branch.
`git push --force`	Overwrites the remote no matter what. **DANGER:** can erase teammates' work.
`git push origin --tags`	Pushes all your tags to the remote.

10. Stash

A drawer for unfinished work when you need to switch tasks right now.

Command	What it does
`git stash`	Shelves your uncommitted changes and gives you a clean working directory.
`git stash push -m "message"`	Same, but with a label so you can recognise it later.
`git stash -u`	Also stashes untracked (new) files.
`git stash list`	Shows all your stashes, numbered like **stash@{0}**.
`git stash show -p`	Shows the changes inside the latest stash.
`git stash pop`	Re-applies the latest stash and removes it from the list.
`git stash apply`	Re-applies the stash but keeps it in the list.
`git stash drop stash@{n}`	Deletes one stash.
`git stash clear`	Deletes all stashes. **DANGER.**
`git stash branch <name>`	Creates a new branch from the stash and applies it there.

11. Cherry-pick

Copy one specific commit from another branch onto the one you are on, such as a hotfix.

Command	What it does
`git cherry-pick <hash>`	Applies that commit's changes here as a new commit.
`git cherry-pick <hash1> <hash2>`	Picks several commits in order.
`git cherry-pick <A>^..<B>`	Picks a whole range of commits, from A through B.
`git cherry-pick -n <hash>`	Applies the changes but does not commit, so you can tweak first.
`git cherry-pick -x <hash>`	Adds a line to the message recording where the commit came from.
`git cherry-pick --continue / --abort`	Finish after resolving a conflict, or cancel the whole operation.

12. Reflog and recovery

Git's safety net. It records every place HEAD has been, even after resets and deleted branches.

Command	What it does
`git reflog`	Lists your recent HEAD movements with their hashes. Find the commit you lost here.
`git reset --hard HEAD@{2}`	Jumps back to where HEAD was two moves ago.
`git branch <name> <hash>`	Creates a branch at a lost commit to rescue it. The safest recovery.
`git switch -c rescue <hash>`	Same idea: create a branch from a lost commit and move onto it.
`git reset --hard ORIG_HEAD`	Undoes the last merge or reset by returning to where HEAD was before it.

Tip: reflog entries are kept for roughly 90 days by default, and they exist only on your own machine.

13. Tags

Permanent labels on commits, usually for releases such as v1.0.0.

Command	What it does
`git tag`	Lists all tags.
`git tag v1.0.0`	Creates a lightweight tag on the current commit.
`git tag -a v1.0.0 -m "Release 1.0.0"`	Creates an annotated tag with a message, author, and date. Preferred for releases.
`git push origin v1.0.0`	Pushes one tag to the remote (tags are not pushed by default).
`git tag -d v1.0.0`	Deletes a local tag.

14. More power tools

Useful extras for debugging and large projects.

Command	What it does
`git bisect start`	Begins a binary search through history to find which commit introduced a bug.
`git bisect bad / git bisect good <hash>`	Mark the current commit as broken, and a known working commit. Git checks out the midpoint for you to test.
`git bisect reset`	Ends the bisect session and returns to your original branch.
`git grep "text"`	Searches for text across all tracked files, faster than normal search.

Command	What it does
`git worktree add ../<folder> <branch>`	Checks out another branch in a separate folder, so you can work on two branches at once without stashing.
`git submodule add <url>`	Embeds another repository inside yours at a fixed commit.
`git archive --format=zip HEAD -o out.zip`	Exports a clean zip of the project without the .git folder.
`git gc`	Cleans up and compresses the repository to save space.
`git diff --stat`	Summarises a diff as a list of files and counts of changed lines.

15. GitHub collaboration and the gh CLI

*Pull requests live on GitHub, not in Git itself. The **gh** tool lets you do GitHub tasks from the terminal.*

Command	What it does
`Fork (button on GitHub)`	Makes your own copy of someone else's repository under your account. Used when you do not have write access.
`git remote add upstream <original-url>`	Adds the original repo as **upstream** so you can pull in its latest changes into your fork.
`git fetch upstream && git merge upstream/main`	Updates your fork's main with the original project's main.
`Pull request (PR)`	A request to merge your branch into another, with a place for discussion, review, and automated checks.
`gh auth login`	Signs the GitHub CLI in to your account.
`gh repo clone <owner>/<repo>`	Clones a GitHub repository using its short name.
`gh repo create <name> --public`	Creates a new repository on GitHub from the terminal.
`gh repo fork --clone`	Forks the current repo and clones your fork.
`gh pr create --title "..." --body "..."`	Opens a pull request from your current branch.
`gh pr list`	Lists open pull requests.
`gh pr checkout <number>`	Downloads someone's pull request branch so you can test it locally.
`gh pr review <number> --approve`	Approves a pull request. Also supports **--comment** and **--request-changes**.
`gh pr merge <number> --squash`	Merges a pull request. Also supports **--merge** and **--rebase**.
`gh issue create --title "..."`	Creates an issue.
`gh run list / gh run watch`	Shows recent GitHub Actions runs and follows one live.

Anatomy of a good pull request: a clear title, a short description of *what* and *why*, how to test it, screenshots for UI changes, a small focused diff, and a linked issue (for example `Closes #12`).

16. GitHub Actions starter

*Automation that runs on GitHub when something happens, such as a push or a pull request. Save this file as **.github/workflows/ci.yml** in your repository.*

```
name: CI

on:                       # when should this run?
  push:
    branches: [main]
  pull_request:
    branches: [main]

jobs:
  test:
    runs-on: ubuntu-latest   # the machine GitHub gives you
    steps:
      - uses: actions/checkout@v4          # get your code
      - uses: actions/setup-node@v4        # install Node
        with:
          node-version: 20
      - run: npm ci                        # install dependencies
      - run: npm test                      # run your tests
```

Key words: workflow is the whole file, **on** is the trigger, **job** is a group of steps on one machine, **step** is one command or reusable action, and **secrets** (Settings, then Secrets) store tokens so they never appear in your code. If a check fails, the PR shows a red cross, and with branch protection it cannot be merged.

17. Team workflows: Git Flow vs trunk-based

Two popular ways for a team to agree on how branches are used.

	Git Flow	Trunk-based development
Main idea	Several long-lived branches with strict roles.	Everyone integrates into one main branch, often several times a day.
Branches	main, develop, feature/*, release/*, hotfix/*	main plus very short-lived feature branches (hours to 2 days).
Release style	Planned releases, versioned, with a stabilising release branch.	Continuous delivery, released straight from main, often behind feature flags.
Best for	Scheduled releases, multiple supported versions, larger or slower-moving teams.	Fast-moving teams with strong automated tests and CI.
Watch out for	Long-lived branches drift apart and cause painful merges.	Needs good tests and discipline, because main must always work.

A simple feature-branch routine (works with either style)

```
git switch main && git pull          # start from the latest main
git switch -c feature/login-form     # one branch per task
# ...edit, then...
git add -p && git commit -m "feat(auth): add login form"
git push -u origin feature/login-form
# open a Pull Request on GitHub, get a review, fix feedback
# merge on GitHub, then clean up:
git switch main && git pull
git branch -d feature/login-form
```

Branch protection (GitHub, Settings, Branches): require a pull request before merging, require approvals, require status checks (your Actions) to pass, and block force pushes to main.

18. Good commit messages (Conventional Commits)

*Format: **type(scope): short summary**. Use the imperative mood, as in "add", not "added". Keep the summary under about 50 characters.*

Command	What it does
`feat: add password reset`	A new feature.
`fix: handle empty cart on checkout`	A bug fix.
`docs: update README setup steps`	Documentation only.
`style: format with prettier`	Formatting changes that do not alter behaviour.
`refactor: split auth service`	Code restructuring with no behaviour change.
`test: add tests for login`	Adding or fixing tests.
`perf: cache product queries`	A performance improvement.
`build: upgrade webpack to 5`	Build system or dependency changes.
`ci: add GitHub Actions workflow`	Changes to CI configuration.
`chore: update .gitignore`	Routine maintenance.
`feat!: drop support for Node 16`	An exclamation mark flags a breaking change.

19. Panic button: common errors and fixes

Do not panic. Almost everything is recoverable if you stop and read the message.

Command	What it does
`I committed to the wrong branch`	Run **git reset --soft HEAD~1**, then **git stash**, **git switch <right-branch>**, **git stash pop**, and commit again.
`Typo in my last commit message`	Not pushed yet? Run **git commit --amend -m "new message"**.
`I forgot a file in my last commit`	Run **git add <file>** then **git commit --amend --no-edit** (before pushing).
`Detached HEAD`	You are on a commit, not a branch. To keep your work run **git switch -c <new-branch>**. To abandon it run **git switch main**.
`Push rejected: non-fast-forward`	The remote has commits you do not. Run **git pull --rebase** (or **git pull**), fix any conflicts, then push again. Do not force push.
`A merge or rebase went badly`	Run **git merge --abort** or **git rebase --abort**. Already finished? Try **git reset --hard ORIG_HEAD**.
`I ran reset --hard and lost commits`	Run **git reflog**, find the hash from before the reset, then **git branch rescue <hash>**.
`I deleted a branch by mistake`	Run **git reflog**, find the branch's last commit, then **git branch <name> <hash>**.
`I need to undo a commit that is already pushed`	Use **git revert <commit>** and push. This is safe because it does not rewrite history.
`I committed .env or a password`	**Rotate the secret immediately**, because it is already exposed. Then remove it from history with **git filter-repo** or BFG, add it to .gitignore, and force push with team coordination.
`File is in .gitignore but still tracked`	Run **git rm --cached <file>** then commit. Ignoring only affects untracked files.
`Please tell me who is on which branch`	Run **git branch -vv** and **git log --oneline --graph --all** to see exactly where everything is.

20. Golden rules

1	Commit small and often. Each commit should do one thing and be easy to describe.
2	Run `git status` before and after almost every command.
3	Pull before you push, and pull before you start new work.
4	Never commit secrets, passwords, API keys, or .env files. Use .gitignore and GitHub secrets.
5	Never rebase or force push shared branches. Use `--force-with-lease` only on your own branch.
6	One branch per task, and delete it after it is merged.
7	Write messages for your future self and your teammates: say *why*, not just what.
8	Review AI-generated commits, PR descriptions, and workflow files before using them, and never paste secrets into an AI tool.
9	When in doubt, `git reflog` remembers. Stay calm and look before you reset.